Local AI Workspace Setup

Internship case study. Original technical notes documenting the setup of existing tools: Ollama, Open WebUI, WSL and Pinokio. Prompt configuration and document retrieval should not be interpreted as evidence of model training. These archived notes are not a maintained installation guide. Original report page numbers are retained.

Portfolio documentation | Iakobi Iakobashvili

Installation des environnement pour IA: Ollama open web ui

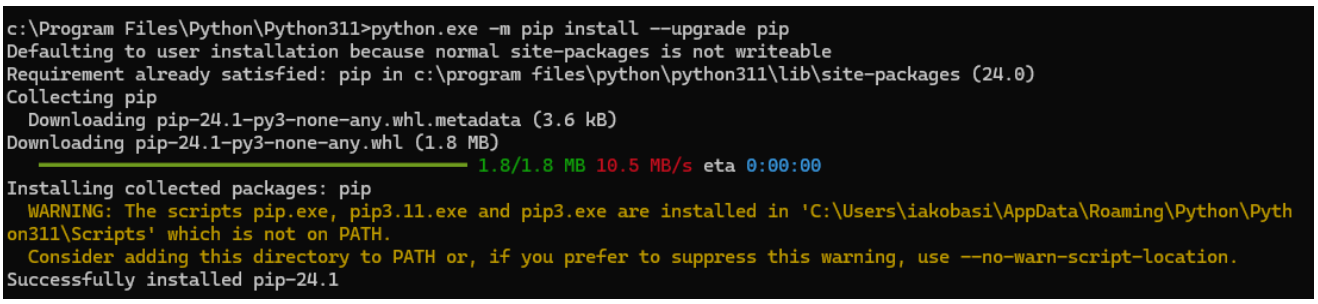
1) Installation wsl: wsl –install

    - Pour télécharger et manipuler les packages/dependencies/données plus facilement.



2) Installer et mettre à jour le pack pip: python.exe -m pip install –upgrade pip

    - Permet pour la suite d'importer et installer des modules importants.



3) Mettre à jour l'environnement ubuntu: apt update

- Pour éviter les erreurs de commandes pas reconnus ou pas à jour.

```
jacobo@DSI-TEST-IA-4:~$ apt update
Reading package lists... Done
E: Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)
E: Unable to lock directory /var/lib/apt/lists/
W: Problem unlinking the file /var/cache/apt/pkgcache.bin - RemoveCaches (13: Permission denied)
W: Problem unlinking the file /var/cache/apt/srcpkgcache.bin - RemoveCaches (13: Permission denied)
jacobo@DSI-TEST-IA-4:~$ sudo apt update
[sudo] password for jacobo:
Get:1 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Hit:2 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:3 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [1583 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:5 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [884 kB]
Get:6 http://security.ubuntu.com/ubuntu jammy-security/universe Translation-en [172 kB]
Hit:7 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Get:8 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [1790 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [326 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1101 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/universe Translation-en [256 kB]
Fetched 6369 kB in 1s (4479 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
16 packages can be upgraded. Run 'apt list --upgradable' to see them.
jacobo@DSI-TEST-IA-4:~$
```

4) Mettre à jour les éléments avancés pour un développeur sur ubuntu: sudo apt upgrade -y

- Pour une utilisation d'ubuntu avancé

```
jacobo@DSI-TEST-IA-4:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
Get more security updates through Ubuntu Pro with 'esm-apps' enabled:
  node-ip ruby-rack
Learn more about Ubuntu Pro at https://ubuntu.com/pro
#
# OpenSSH CVE-2024-6387 fix is available for all affected Ubuntu releases.
# RegreSSHion: Possible RCE Due To A Race Condition In Signal Handling.
# For more details see: https://ubuntu.com/blog/ubuntu-regresshion-security-fix
#
The following NEW packages will be installed:
  ubuntu-pro-client
The following packages have been kept back:
  python3-update-manager update-manager-core
The following packages will be upgraded:
  libpython3.10 libpython3.10-dev libpython3.10-minimal libpython3.10-stdlib python3.10 python3.10-dev
  python3.10-minimal python3.10-venv tzdata ubuntu-advantage-tools ubuntu-minimal ubuntu-pro-client-l10n
  ubuntu-standard ubuntu-wsl
14 upgraded, 1 newly installed, 0 to remove and 2 not upgraded.
8 standard LTS security updates
Need to get 13.2 MB of archives.
After this operation, 143 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 python3.10-dev amd64 3.10.12-1~22.04.4 [508 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libpython3.10-dev amd64 3.10.12-1~22.04.4 [4763 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libpython3.10 amd64 3.10.12-1~22.04.4 [1949 kB]
```

5) Télécharger le module principal pour le fonctionnement des LLM: pip install transformers

   - Permet d'importer une bibliothèque "transformers" pour les LLM.
   - Cette commande n'est pas obligatoire si nous utilisons un logiciel comme docker ou pinokio, car ces logiciels important et installent les modules nécessaires automatiquement pendant la mise en page de l'interface web.
   - Nous avons besoin de manuellement installer de tels modules si nous codons une llm sur python.

```
jacobo@DSI-TEST-IA-4:~$ pip install transformers
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: transformers in ./.local/lib/python3.10/site-packages (4.43.0.dev0)
Requirement already satisfied: requests in ./.local/lib/python3.10/site-packages (from transformers) (2.32.3)
Requirement already satisfied: filelock in ./.local/lib/python3.10/site-packages (from transformers) (3.15.4)
Requirement already satisfied: tokenizers<0.20,>=0.19 in ./.local/lib/python3.10/site-packages (from transformers) (0.19
.1)
Requirement already satisfied: safetensors>=0.4.1 in ./.local/lib/python3.10/site-packages (from transformers) (0.4.3)
Requirement already satisfied: numpy<2.0,>=1.17 in ./.local/lib/python3.10/site-packages (from transformers) (1.26.4)
Requirement already satisfied: huggingface-hub<1.0,>=0.23.2 in ./.local/lib/python3.10/site-packages (from transformers)
 (0.23.4)
Requirement already satisfied: pyyaml>=5.1 in /usr/lib/python3/dist-packages (from transformers) (5.4.1)
Requirement already satisfied: tqdm>=4.27 in ./.local/lib/python3.10/site-packages (from transformers) (4.66.4)
Requirement already satisfied: packaging>=20.0 in ./.local/lib/python3.10/site-packages (from transformers) (24.1)
Requirement already satisfied: regex!=2019.12.17 in ./.local/lib/python3.10/site-packages (from transformers) (2024.5.15
)
Requirement already satisfied: fsspec>=2023.5.0 in ./.local/lib/python3.10/site-packages (from huggingface-hub<1.0,>=0.2
3.2->transformers) (2024.5.0)
Requirement already satisfied: typing-extensions>=3.7.4.3 in ./.local/lib/python3.10/site-packages (from huggingface-hub
```

6) Télécharger l'interface API intégrée "ollama" sur wsl:
   curl -fsSL https://ollama.com/install.sh | sh

   - C'est pour appliquer l'interface API de base afin de permettre de construire une interface web à la suite.

```
jacobo@DSI-TEST-IA-4:~$ curl -fsSL https://ollama.com/install.sh | sh
>>> Downloading ollama...
######################################################################## 100.0%##O#-#
######################################################################## 100.0%
>>> Installing ollama to /usr/local/bin...
>>> Creating ollama user...
>>> Adding ollama user to render group...
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
>>> Enabling and starting ollama service...
Created symlink /etc/systemd/system/default.target.wants/ollama.service → /etc/systemd/system/ollama.service.
>>> Nvidia GPU detected.
>>> The Ollama API is now available at 127.0.0.1:11434.
>>> Install complete. Run "ollama" from the command line.
```

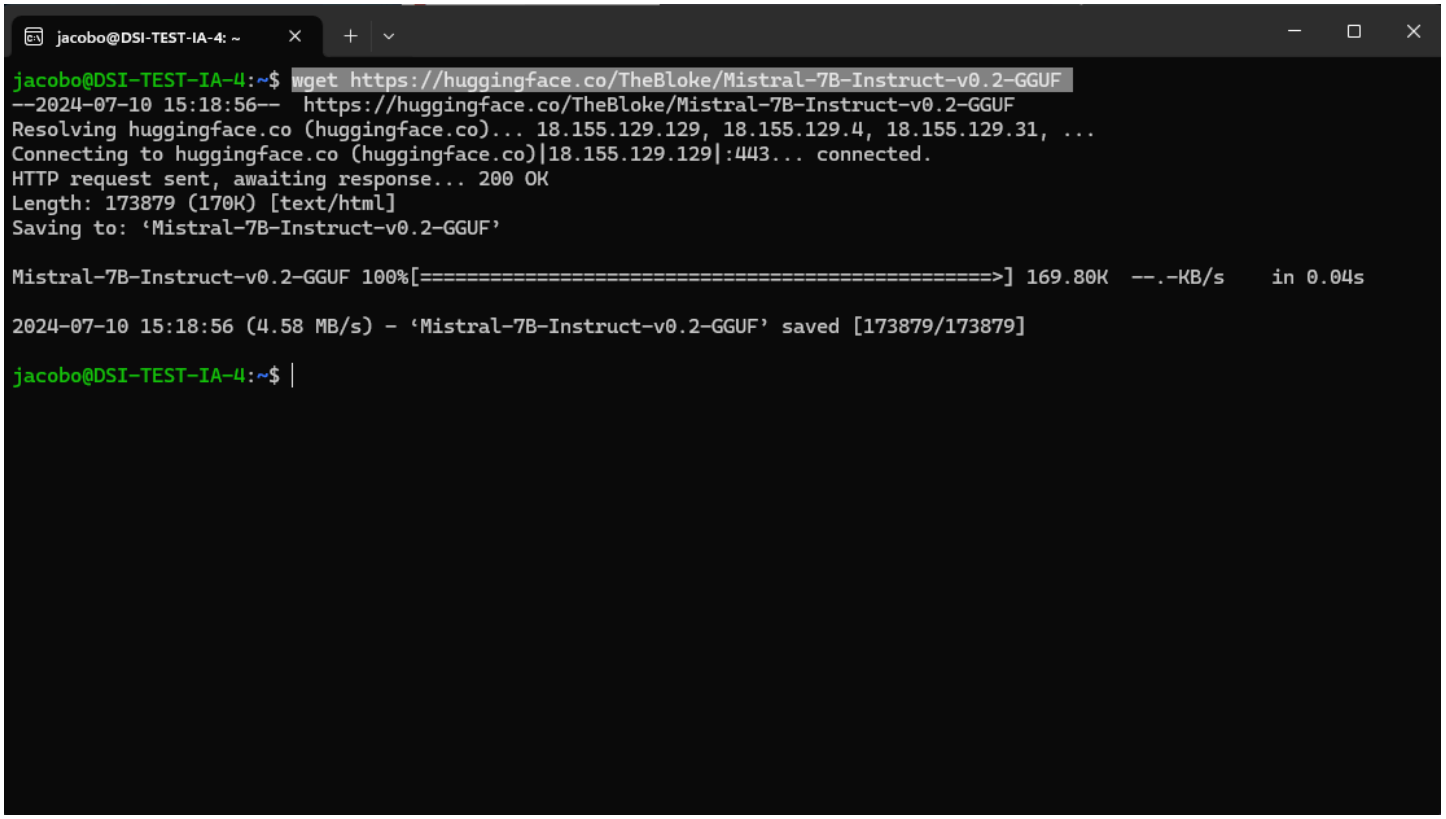
7) Télécharger un modèle llm dans ollama: ollama run < modèle >

- Dans ce cas, le modèle souhaité doit obligatoirement provenir de la bibliothèque principale de site web officiel de Ollama, car la commande envoie une requête sur ce site web pour rechercher un modèle.
- Ce sont des modèles classiques officiels non modifiés (pré-entrainés).



8) Téléchargement d'un modèle entrainé ou pré-entrainé, non-officiel ou officiel, de site web "hugging face": exemple →
wget https://huggingface.co/TheBloke/CapybaraHermes-2.5-Mistral-7B-GGUF

- La commande "wget" permet de télécharger un fichier quelconque depuis wsl.
- Il faut prendre en compte le chemin de téléchargement, il faut d'abord se rendre sur le chemin souhaité afin d'y télécharger un modèle. Commande de chemin: cd <chemin>
- Remarque: cela ne peut pas être utilisé pour l'interface web ollama: open web-ui, car comme constaté cette interface prend en compte que ses modèles intégrés dans sa propre bibliothèque.



9) Installations des packages avec un fichier: exemple
pip install -r requirements.txt

```
C:\WebUI\extensions\superboogav2>pip install -r requirements.txt
Defaulting to user installation because normal site-packages is not writeable
Collecting beautifulsoup4==4.12.2 (from -r requirements.txt (line 1))
  Downloading beautifulsoup4-4.12.2-py3-none-any.whl.metadata (3.6 kB)
Collecting chromadb==0.4.24 (from -r requirements.txt (line 2))
  Downloading chromadb-0.4.24-py3-none-any.whl.metadata (7.3 kB)
Collecting lxml (from -r requirements.txt (line 3))
  Downloading lxml-5.2.2-cp311-cp311-win_amd64.whl.metadata (3.5 kB)
Collecting optuna (from -r requirements.txt (line 4))
  Downloading optuna-3.6.1-py3-none-any.whl.metadata (17 kB)
Collecting pandas==2.0.3 (from -r requirements.txt (line 5))
  Downloading pandas-2.0.3-cp311-cp311-win_amd64.whl.metadata (18 kB)
Collecting posthog==2.4.2 (from -r requirements.txt (line 6))
  Downloading posthog-2.4.2-py2.py3-none-any.whl.metadata (2.0 kB)
Collecting sentence_transformers==2.2.2 (from -r requirements.txt (line 7))
  Downloading sentence-transformers-2.2.2.tar.gz (85 kB)
     ---------------------------------------- 86.0/86.0 kB 2.4 MB/s eta 0:00:00
  Installing build dependencies ... done
```

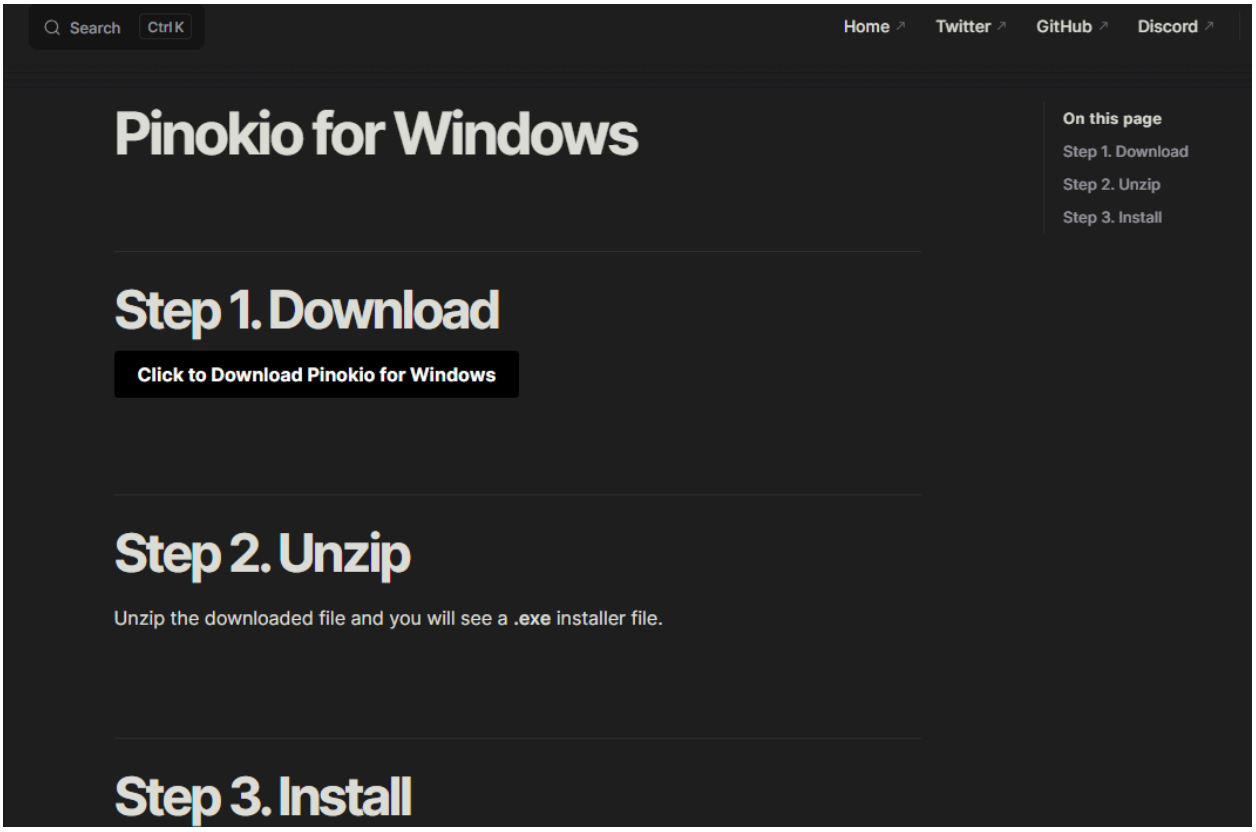
- “-r ” est pour “requirements”

Installation interface web: Open Webui

1) Téléchargement Pinokio ou Docker:

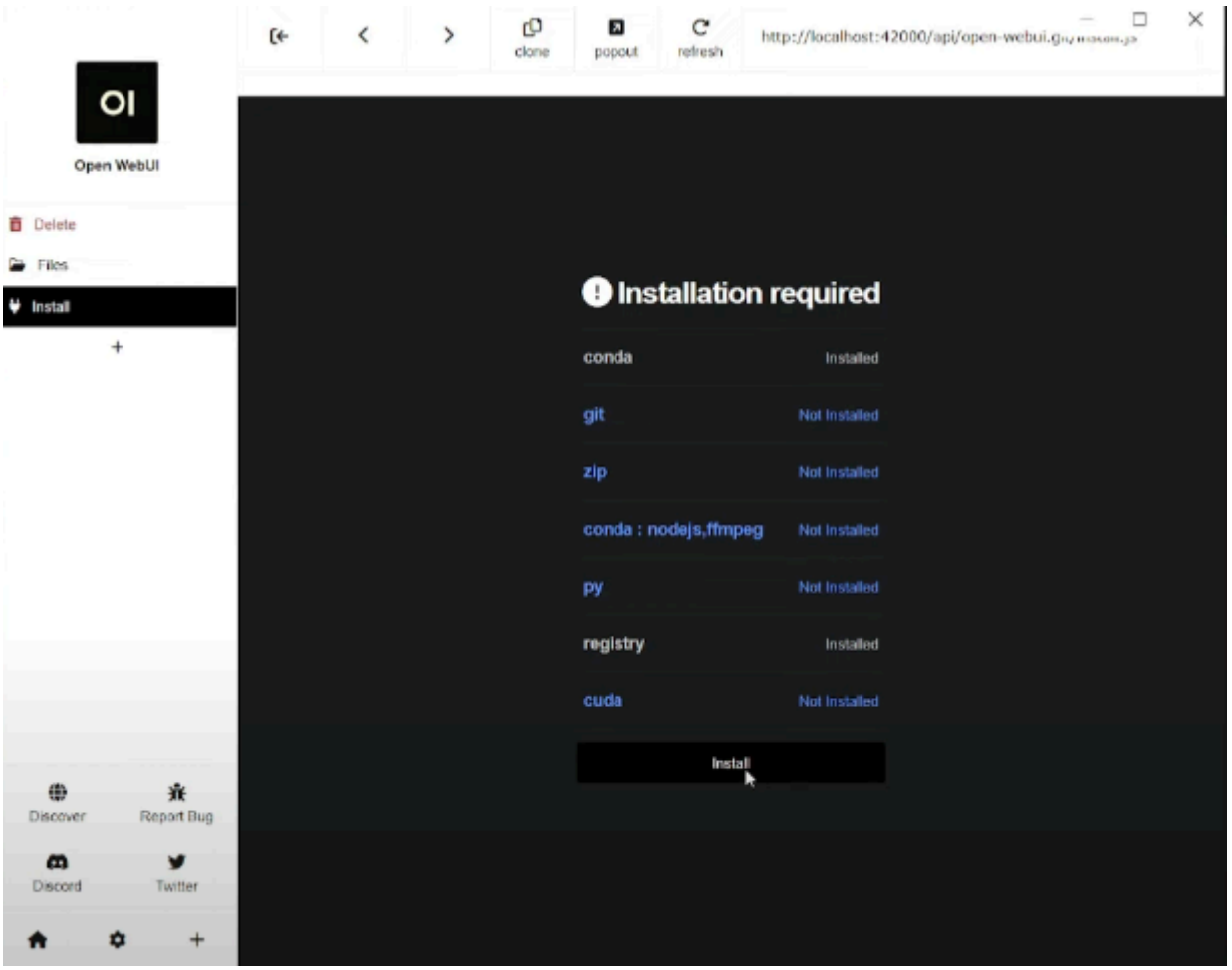
   - Pinokio est un navigateur d'installation des interfaces IA.
   - Docker est un logiciel de création et de déploiement des applications et des interfaces conteneurisées localement.
   - J'ai choisi Pinokio, car c'est plus rapide comme solution.
   - L'installation et l'utilisation de Docker demande souvent les mots de passe admin. (Peut être problématique pour un stagiaire)

Pinokio: https://docs.pinokio.computer/download/windows.html
Docker: www.docker.com



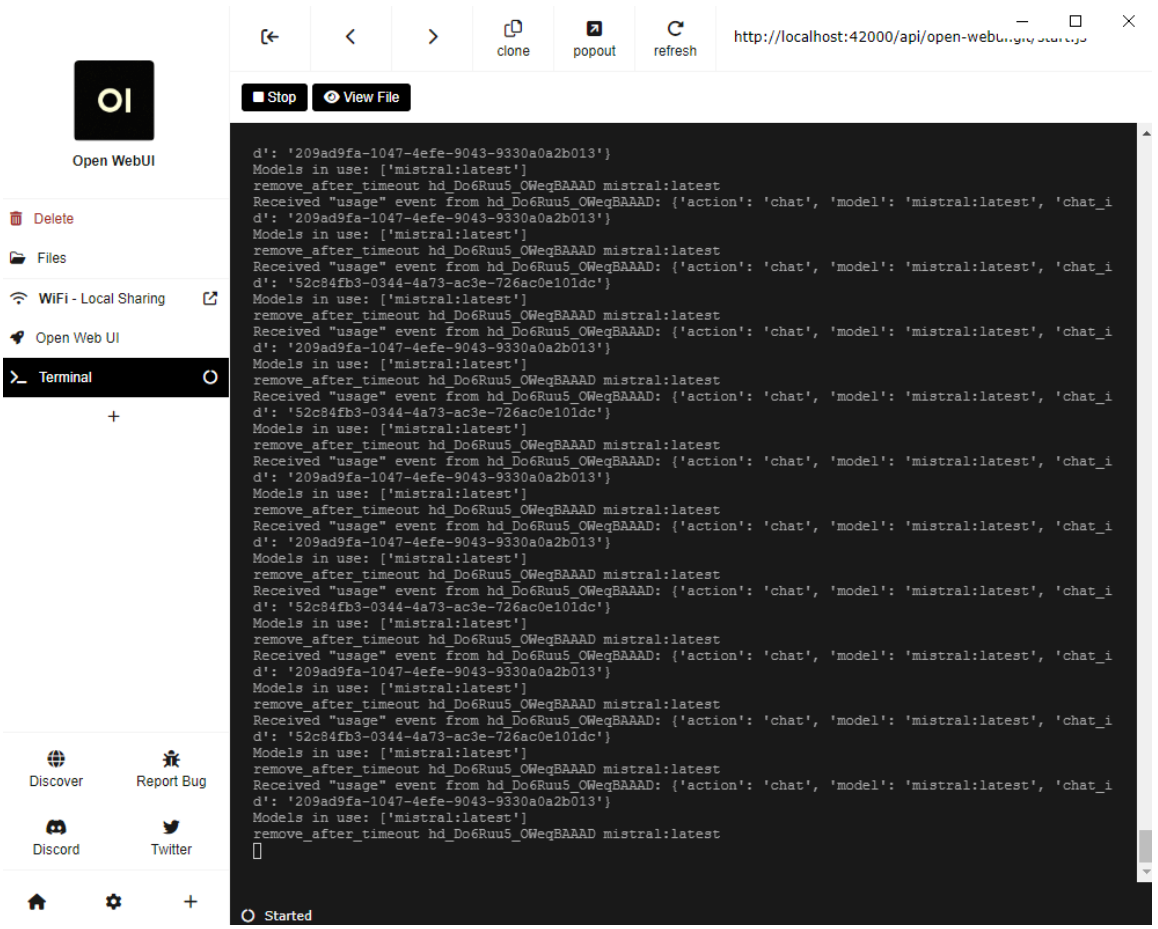
2) Installation Pinokio:

- Pinokio va automatiquement installer et sauvegarder l'environnement développeur et modules importants.



3) Mettre en place open web ui en locale:

- Dans Pinokio → “Discover” il faut chercher l’interface open web ui et la télécharger.
- Il s’agit d’un “git”
- Quand l’installation est terminé il faut la lancer par “start”
- Une fois lancée, pinokio va générer un lien “localhost” pour accéder à l’interface en web.
- Ouvrir l’interface web dans un navigateur
- Lien par défaut localhost:8000

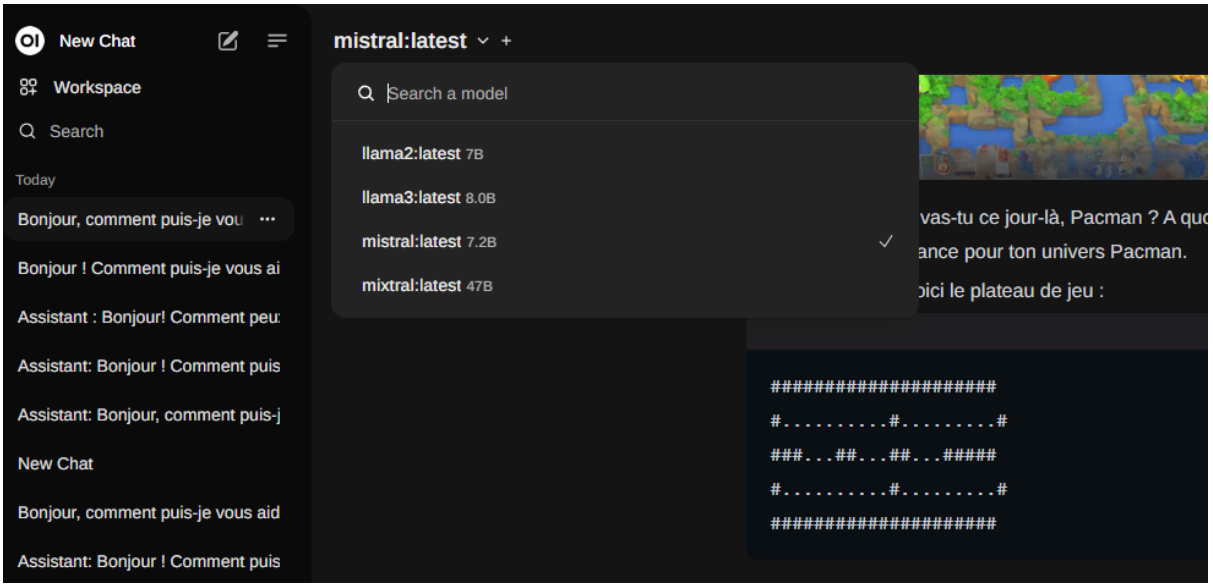


Chargement des modèles LLM:

1) Dans wsl entrer la commande suivante:

   - ollama run <modèle souhaité>
   - ollama run mistral
   - Rafraîchir la page d'interface web pour prendre en compte la modification.



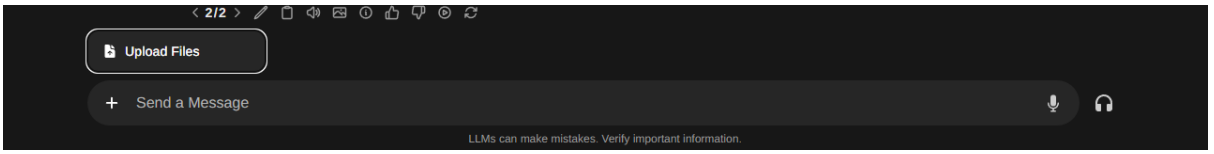


Les pages / fonctionnalitées / options importants

1) Input fichier et vocal:

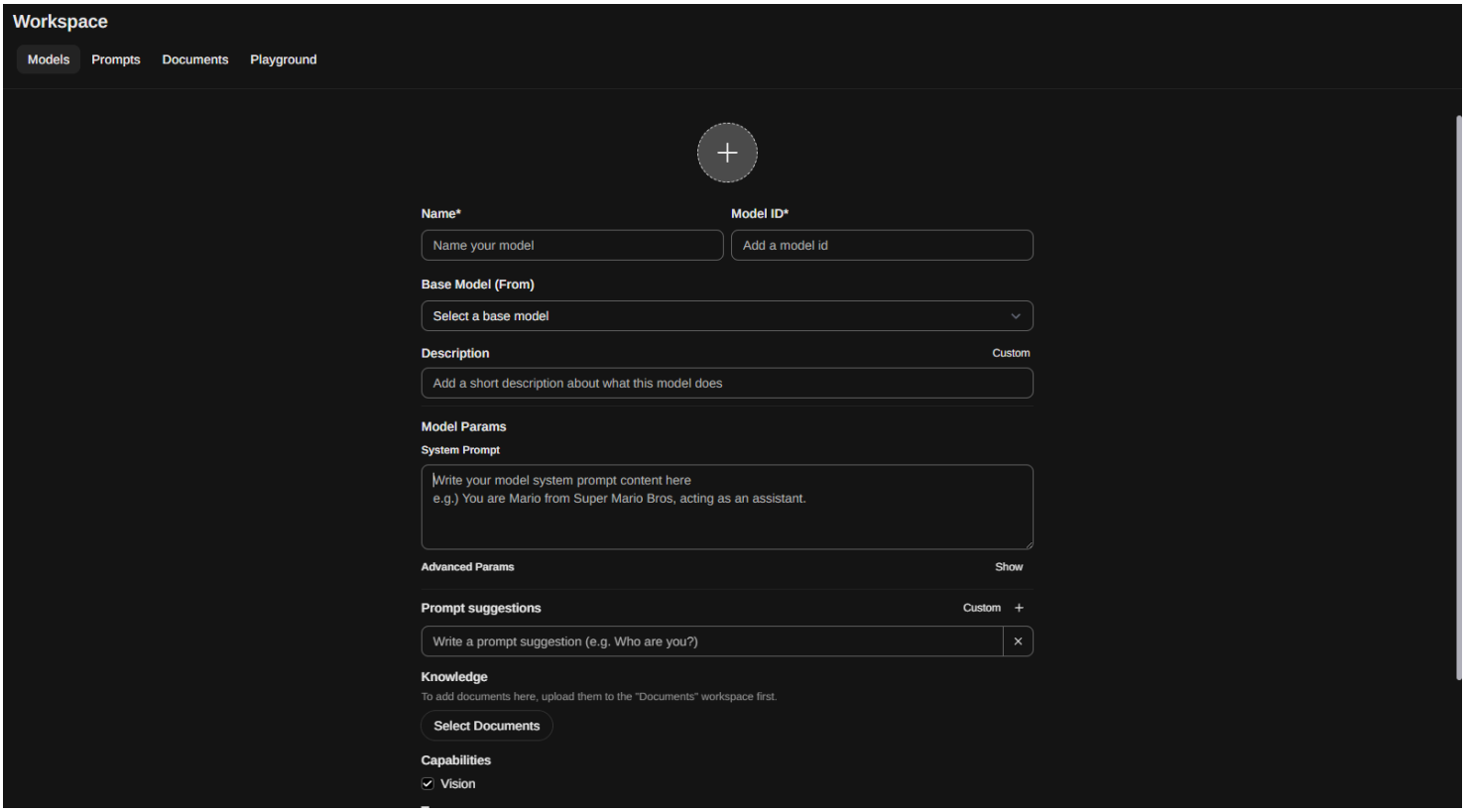
   Exemple d'utilisation:

   - Input fichier: Insérer les documents et poser des questions
   - Input vocal: Enregistrer un vocal pour discuter avec l'IA



2) Entraîner le modèle localement: dans workspace

   - Créer une caractère du modèle
   - Tester des prompts
   - Importer les documents en tant que données

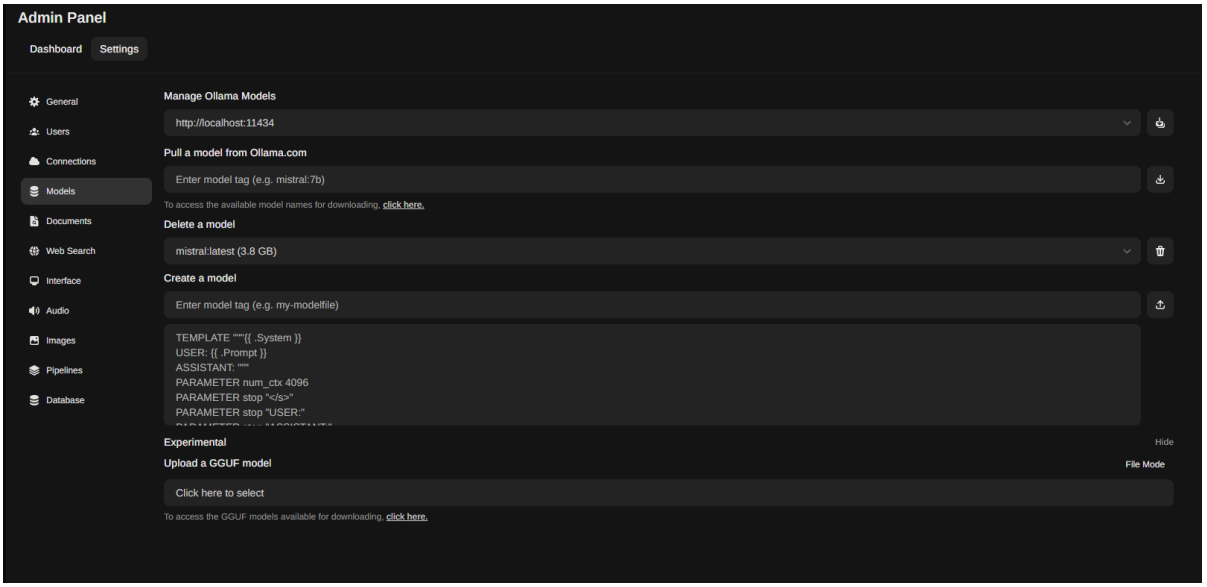


- Tester les retours

3) Aller dans settings → admin settings pour plus de paramètres.

Manipulation modèles: dans paramètres → modèles

- Télécharger le modèle depuis l'interface
- Créer un modèle ou importer un modèle entraîné



4) Embedding et RAG dans paramètres → documents:

    - Donner des contextes détaillées pour plus de précisions

5) Enregistrer un prompt par défaut pour un modèle souhaité: dans paramètres → interface

    - Agit comme une consigne ou context lors la génération de réponses

6) Intégration pipelines dans paramètres → pipelines:

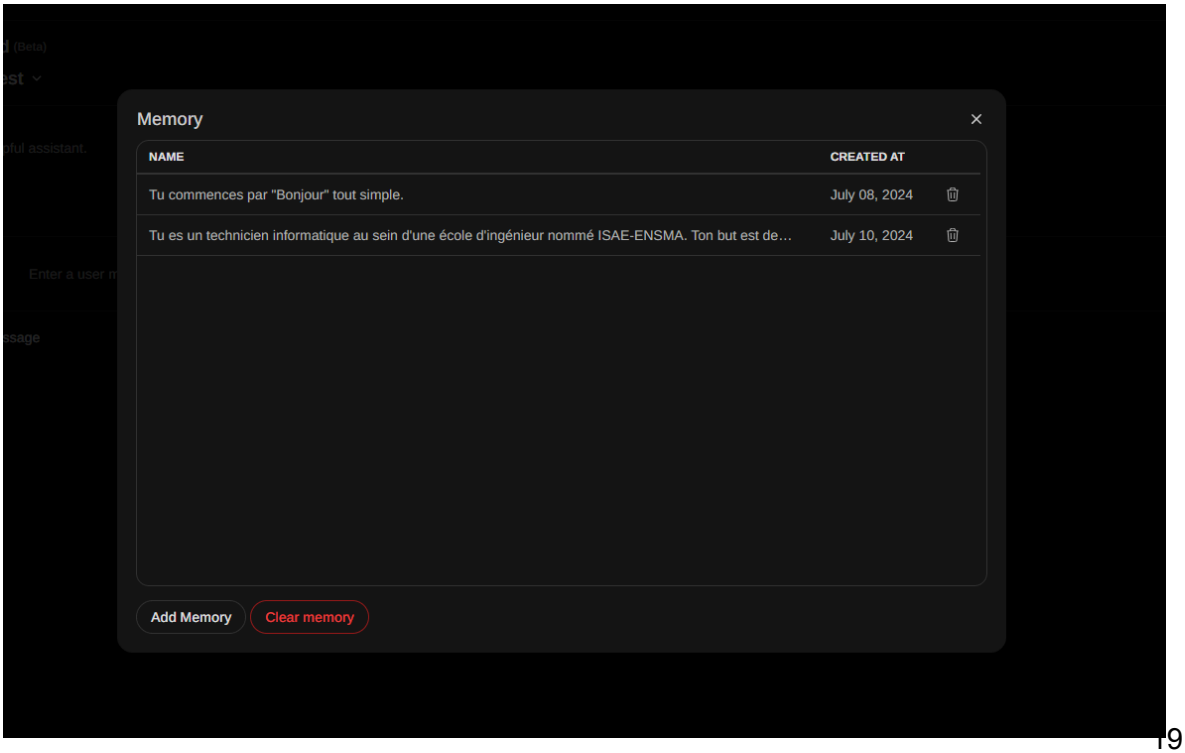
    - Permet aux LLM d'améliorer l'efficacité de calculs et de la recherche dans les données.

7) Données régénérées: dans paramètres → databases:

    - Exporter les données travaillées

8) Mettre une place une mémoire personnalisée: dans paramètres → personnalisation (avant d'aller dans "admin settings")

    - Permet aux LLM de prendre en compte une information précisée.



9) Intégrer des documents en tant que données locales: dans workspace → documents

    - Permet aux LLM de d'aller chercher les données dans les documents partagées en plus de ses propres données.

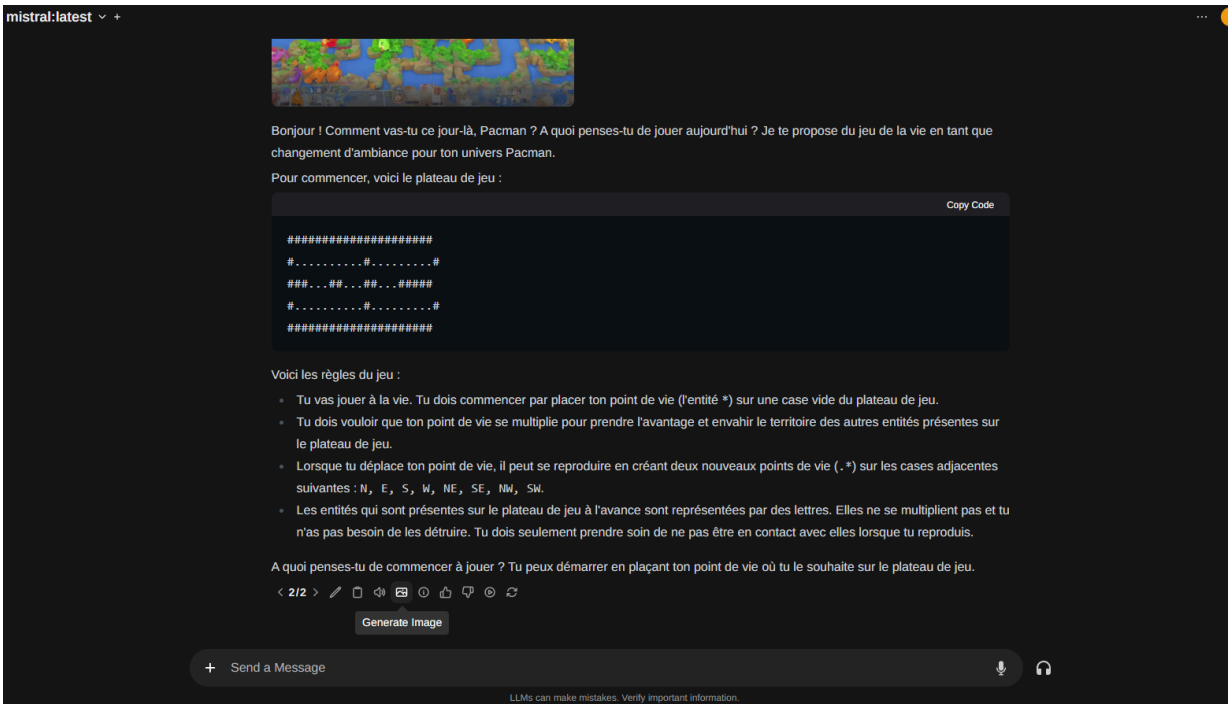
Mettre en place l'API de "stable diffusion" pour images

Une sortie d'image générée: donner un texte et générer une image selon la description.

Il faut installer "stable diffusion" pour immages, générer le lien de son API et l'enregistrer dans les paramètres de open web ui: paramètres → admin settings → images

Dans wsl:

1) Téléchargement: wget https://github.com/AUTOMATIC1111/stable-diffusion-webui
2) Mettre en place: bash webui.sh --listen --api (il faut d'abord aller dans le chemin où se trouve "webui.sh")
3) Copier le lien API d'interface stable diffusion. Coller dans open webui



(paramètres → admin settings → images)

4) Il y a la possibilité de lancer l'interface web de stable diffusion à part avec son lien localhost généré, pour avoir accès et manipuler ses paramètres.

